

PCCST503 – Machine Learning

Assignment 1

Design of a Safe Semantic Planner in a Finite Cartesian State Space

Algorithm: D* Lite

Department of Computer Science and Engineering

Submitted by: Dilshath PK

1. Abstract

This project implements a Safe Semantic Planner for a finite Cartesian state space using a D* Lite-inspired incremental graph-search approach. The planner finds a path from an initial state to a goal while avoiding explicitly marked bad states. Transition cost, reliability, availability and Cartesian state embeddings are represented in the planning model. The implementation also demonstrates replanning for dynamic transition changes, goal updates and newly added shortcuts.

2. Problem Definition

The planning environment consists of a finite set of states embedded in a Cartesian space. A planning problem contains an initial state, goal state, bad states and directed transitions. Each transition has a cost, safety value, reliability value and availability flag. A valid solution must reach the goal without visiting a bad state. The implementation evaluates total path cost, minimum Euclidean safety distance, cumulative reliability, explored states and planning time.

3. State Representation

Each state is represented by a 64-bit identifier and a vector of Cartesian coordinates. The test environment contains $S=(0,0)$, $A=(2,2)$, $B=(4,3)$, $C=(2,-2)$, $G=(6,0)$, and $X=(4,1)$. X is designated as a bad state and is therefore excluded from valid paths.

4. Transition Representation

A transition stores an identifier, source state, destination state, cost, safety score, reliability and availability flag. During planning, unavailable transitions are ignored and transitions leading to bad states are rejected.

5. Data Structures

The implementation uses C++ classes for State, Transition, PlanningProblem and PlanningResult. Hash maps store g and rhs values, while a priority queue supports shortest-path computation. Vectors store states, transitions and resulting paths. An unordered set is used during path reconstruction to prevent loops.

6. D* Lite Planning Approach

The planner computes shortest-path information backward from the goal and then reconstructs a path from the initial state. The goal has rhs value zero, and predecessor states are updated using available transitions. For each current state, the planner selects an available transition with the lowest transition cost plus the stored cost-to-go value. This allows the planner to recompute a solution when the environment changes.

7. Heuristic Function

Cartesian embeddings allow Euclidean distance to be used as a heuristic or spatial measure. For states s and g , $h(s,g) = \sqrt{\sum((s_i - g_i)^2)}$. This provides a geometric estimate of distance between states and is consistent with the Cartesian representation.

8. Safety Computation

For every state on a successful path, the distance to every bad state is calculated. The safety score is the minimum of these distances. Thus, a larger minimum safety distance means the complete path maintains

a larger clearance from bad states. Bad states themselves are never allowed into a valid path.

9. Cost and Reliability

Total path cost is the sum of transition costs. Cumulative reliability is calculated as the product of the reliability values of all transitions on the selected path. These metrics are reported with every experiment. The current implementation prioritizes safe reachability and minimum transition cost, while safety distance and reliability are evaluated as additional performance measures.

10. Dynamic Environment and Replanning

The planner supports dynamic changes by constructing a `PlanningProblem` from the current environment and recomputing path information. Test Case 4 disables a transition and obtains an alternative route. Test Case 5 changes the goal from G to B and produces a new path. Test Case 6 adds a direct S-to-G shortcut and discovers the improved route. A production implementation could preserve and incrementally update D^* Lite structures instead of recomputing all values.

11. Complexity

For the implemented graph-search computation, the priority-queue shortest-path process is approximately $O((V+E) \log V)$, where V is the number of states and E is the number of available transitions. Path reconstruction is $O(V)$ in the worst case. Safety evaluation can take $O(VB)$ for B bad states. Memory usage is $O(V+E)$ for graph information and search structures.

12. Experimental Test Cases

Six test cases were executed. They cover basic reachability, bad-state avoidance, cost/safety behavior, transition removal, goal updates and transition addition.

Test	Success	Bad States	Cost	Min Safety	Reliability	Explored	Time (ms)
1	1	0	11	2.23607	0.9506	5	0.0218
2	1	0	11	2.23607	0.9506	5	0.0124
3	1	0	6	2.00000	0.81225	5	0.0341
4	1	0	11	2.23607	0.9506	4	0.0126
5	1	0	9	2.00000	0.85500	3	0.0108
6	1	0	4	2.23607	0.99000	5	0.0144

13. Results and Discussion

All six experiments achieved successful goal completion and visited zero bad states. Test 4 demonstrates that disabling a transition still permits an alternative valid route. Test 5 successfully adapts to a changed goal. Test 6 demonstrates the benefit of a newly inserted shortcut: the path cost decreases from 11 to 4 and reliability increases to 0.99. Planning times are very small for this test graph, ranging from 0.0082 ms to 0.0341 ms in the recorded experiments.

14. Conclusion

The Safe Semantic Planner successfully demonstrates graph-based planning in a finite Cartesian state space. The implementation reaches goals safely, avoids bad states, calculates cost and safety metrics, and responds to dynamic changes. The six experimental cases validate the required behaviors and

provide measurable performance results.

15. User Manual

On Windows with VS Code, compile using: `g++ -std=c++17 src/main.cpp -o planner`. Run using: `.\planner.exe`. The program prints all six test cases and writes experimental data to `results/results.csv`. The source code is contained in `src/main.cpp`.

16. References

1. P. Hart, N. Nilsson and B. Raphael, A Formal Basis for the Heuristic Determination of Minimum Cost Paths, IEEE Transactions on Systems Science and Cybernetics, 1968.
2. S. Koenig and M. Likhachev, D* Lite, AAAI Conference on Artificial Intelligence, 2002.
3. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach.